Vigil — Presentation Script

Software Engineering · Final project presentation · Vatsal Agrawal (24BAI1355)

About 15 minutes of talk and live demo, then questions. Lines in *italics* are what you say. Normal text is what you do. If you're short on time, the ⏩ markers show what to skip.

Part 0 — Morning checklist (do this 15 minutes before)

1. **Start Docker Desktop.** Wait until it says Engine running.
2. **Reset the demo** in a terminal at the repo folder: `cd "C:\Users\Vatsa\OneDrive\Documents\all about AI\vigil\vigil" git pull powershell -ExecutionPolicy Bypass -File scripts\demo-reset.ps1` It takes about 3–4 minutes and must end with **"All checks passed. Ready to present."** If anything prints `[FAIL]`, the line under it says how to fix it.
3. **Open these browser tabs, in this order:**

#	Tab	URL
1	GitHub repo	https://github.com/vatsal-agra/vigil
2	Project board (by MoSCoW)	https://github.com/users/vatsal-agra/projects/2/views/1?groupedBy%5BcolumnId%5D=374818031
3	Vigil dashboard, **signed in**	http://localhost:3000 (demo@vigil.dev / vigil-demo-2026)
4	Public status page, **in a private/incognito window**	http://localhost:3000/status/demo
5	API docs	http://localhost:8000/docs
6	GitHub Actions	https://github.com/vatsal-agra/vigil/actions
7	Releases	https://github.com/vatsal-agra/vigil/releases

1. **Keep a terminal open** at the repo folder, with a large font (Ctrl + mouse-wheel).
2. **Have the three PDFs ready** as backup: `Vigil_Digital_Assignment_1.docx` (repo root), `docs/design/Vigil_Software_Design_Document.pdf`, and `docs/report/Vigil_Project_Report.pdf`.
3. Close Slack, WhatsApp and notifications. Turn on Do Not Disturb.

Part 1 — Opening (30 s)

"Good morning. My project is Vigil : uptime and API monitoring that explains its own outages. I'll cover the problem, how I planned it, how it's designed, then show it running live, and finish with how I tested and released it."

Part 2 — The problem and the vision (1.5 min) · Review 1

Show **tab 1** (the GitHub repo) and scroll slowly through the README's vision section.

"Anyone who ships a website or an API has the same problem. It breaks at 3 a.m., and they find out at 11 a.m. from a user asking 'is your site down?'"

"The existing options each fall short. Pingdom and Datadog are priced for companies. Free tiers check every five minutes and keep days of history. A cron job with curl has no history at all. Vigil is self-hosted. It checks your services on a schedule, opens an incident by itself when something breaks, closes it when it recovers with a one-line explanation of what happened, and publishes a public status page. It all starts with one command."

"I designed for three personas: Rhea, a solo founder who needs to know before her customers do; Arjun, a backend engineer who needs latency history and alerts he can trust; and Meera, a freelancer who needs unlimited monitors for her clients."

Part 3 — Planning: 25 user stories and MoSCoW (1 min) · Review 1

Show **tab 2** (the project board, grouped by MoSCoW).

"I broke the product into 25 user stories across six epics, each with acceptance criteria and story points. They live as GitHub Issues on this Projects board, and I prioritised them with MoSCoW: 16 Must-haves, 6 Should, 2 Could and 1 Won't-have for this release. Everything on this board, the issues, labels and fields, was created by a script from one source file, so the backlog is reproducible."

"The Must-haves are the core loop: sign in, add a URL, the system checks it, and the system tells you when it broke. Remove any one of them and it isn't a monitoring tool any more."

Part 4 — Design (2 min) · Review 2

Open `docs/design/` on GitHub, or the Software Design Document PDF, and show the **module and layer diagram** (Figure 2).

"The architecture is a layered monolith deployed as five containers: a Next.js frontend, a FastAPI backend, a separate probe worker, PostgreSQL, and Redis as a cache."

"Two decisions matter most. First, the probe worker is a separate container. Checking a website is slow, blocking work; a dead site can hold a thread for eight seconds. If that ran inside the API, the dashboard would slow down during an outage, exactly when people are refreshing it. Second, Redis is a cache-aside layer for the status page, because that page gets its traffic spike exactly when something is failing."

"I followed four design principles. Abstraction: the frontend has one API client and no page ever calls fetch directly. High cohesion: the probe engine is three single-purpose functions. Low coupling: the API and the worker never call each other; they only share the database and the cache-key names. Modularity: each file has one job, and dependencies only point downward."

⏩ Optional: show the **monitor state machine** (Figure 4): "One failed check is noise. Two in a row open an incident, and the first success closes it."

Part 5 — Live demo (5 min) · the product working

Step 5.1: break something first, because detection takes about 1–2 minutes. Show **tab 3** (the dashboard). Click **Add monitor**, then: - Name: `Payments API` - URL: `https://httpbin.org/status/503` ← this URL always answers "503 Service Unavailable" - Click **Start watching**.

"I've just added a service that is broken on purpose. It shows as PENDING. I'll come back to it in a minute: I'm not going to touch it, and Vigil should notice on its own."

Step 5.2: tour the dashboard. "This is the dashboard: every monitor on one screen with status, 24-hour uptime and response time. Status is always shown as a word as well as a colour, for accessibility. It refreshes every 15 seconds."

Click **Legacy invoice worker** (the red, DOWN one). "This is the detail page: a 24-hour response-time chart, with failed checks as red markers, and the raw check log in plain English: 'Expected 200, got 503', not an exception name."

Click **Back to dashboard**.

Step 5.3: the public status page. Show **tab 4** (the private window). "This is what a customer sees, with no login. It shows only the monitors marked public, an overall banner based on the worst one, and never any internal URLs or errors. It's served from the Redis cache."

Step 5.4: the API. Show **tab 5** (Swagger). "The backend is a documented REST API. FastAPI generates this page from the code, with every endpoint and schema."

Step 5.5: back to the broken monitor (by now 1–2 minutes have passed). Show **tab 3**, refresh, and scroll to **Recent incidents**. "Payments API is now DOWN, and an incident has opened by itself: 'Expected 200, got 503'. It waited for two consecutive failures before opening it. That's the flap guard, so one blip doesn't wake anyone at 3 a.m."

> If it still says PENDING, keep talking (go to Part 6) and come back. It needs two probe cycles.

Step 5.6: pause and delete (US-12). Click **Payments API**, then **Pause checks**: "Paused: the worker skips it now." Click **Stop watching**: "It asks for confirmation, because deleting removes its history." Click **OK**.

⏩ **Step 5.7: resilience** (only if time allows). In the terminal:

```
docker compose stop cache
```

Refresh the dashboard twice. "Redis is down, and the app still works: reads fall through to PostgreSQL. The first request is slow while it detects the outage. After that a circuit breaker skips Redis for 30 seconds, so the rest are instant."

```
docker compose start cache
```

Part 6 — Testing (2 min) · Review 3

In the terminal, run:

```
docker compose exec api python -m pytest -m regression -v
```

"I test at four levels. 83 backend tests run against a real PostgreSQL and Redis, not mocks: 54 unit, 22 integration and 7 regression tests. There are also 6 end-to-end tests that drive a real browser. Coverage is 93%. These are the regression tests: each one is named after a bug I fixed, so it can never come back. R1 is a real bug from Review 2, where the dashboard said 'zero open incidents' while a service was down."

⏩ Optionally run all of them: `docker compose exec api python -m pytest -q` (about 15 s).

Open the report (`docs/report/Vigil_Project_Report.pdf`), section 1.3, the mutation table.

"Coverage only tells you a line ran, not that a test would catch a bug in it. So I used mutation testing. The tool mutmut makes 120 small bugs in my business logic, like flipping greater-or-equal to greater-than or times to divide, and checks whether a test fails. My first score was 77.5%. The surviving mutants showed me real gaps, for example that I never pinned latency to an exact value, and after I fixed them it's 99.2%. The only survivor is an 'equivalent' mutant: a change that can't alter behaviour, so no test could ever catch it."

"Writing the browser tests straight from the acceptance criteria also found two real gaps: you couldn't pause a monitor from the UI, and delete didn't ask for confirmation. Both are fixed."

Part 7 — Version management and build (1 min) · Review 3

Show **tab 6** (Actions). "Since the initial commit, every change has gone through a pull request using GitHub Flow. CI runs the unit, integration and regression tests against real database containers, builds the frontend and builds both Docker images. Nothing merges unless it's all green."

Show **tab 7** (Releases). "Each review is a semantic-version release: v0.1.0 for the walking skeleton, v0.2.0 for the design, and v0.3.0 for testing. Commits follow the Conventional Commits format and reference the user story they implement."

Part 8 — Close (30 s)

"To sum up: Vigil detects outages by itself, explains them in plain English, and publishes a status page. It's designed in clean layers, tested down to mutation level, and the whole thing runs from one `docker compose up`. Next I'd add email and Slack alerts, which are the Should-have stories still open. Thank you, I'm happy to take questions."

Part 9 — Likely questions and answers

Why not microservices? One developer and one semester. Microservices add network hops, distributed failures and deployment overhead with no benefit at this size. I split out only the worker, because blocking probes must not slow the API. The modules are already layered, so I could split a service out later along an existing seam.

Why Redis? Isn't PostgreSQL enough? For correctness, yes. Redis is there for load. The public status page spikes exactly when something is failing; with a 20-second cache, a thousand visitors cause about three database queries a minute. The worker deletes the affected cache keys on every write, so data is never staler than one probe cycle.

Why two failures before an incident? One failed check is usually network noise. Alerting on single failures fills the channel with false alarms until everyone mutes it. The threshold is a setting (`failure_threshold`), not hard-coded.

Why PostgreSQL rather than SQLite? The API and the worker write at the same time. SQLite's single-writer lock makes that a correctness problem, not just a performance one.

How do you keep users' data separate? Every query filters by the owner. Another user's monitor returns **404, not 403**, so you can't even confirm it exists. Passwords are salted PBKDF2-SHA256 hashes, and sessions are signed JWTs that expire after 24 hours.

What is mutation testing, in one line? Deliberately putting small bugs into the code and checking that the tests catch them. It measures how strong the tests are, not just how much code they run.

What's an equivalent mutant? A change that can't affect behaviour. Mine is `.strip("-")` becoming `.strip("XX-XX")` on text that's already lower-case, where an 'X' can never appear. No test can tell the two apart, so it's correctly left alive.

What isn't covered by tests? About 7%: the start-up wait loop and the demo seed script, which run once at container start, plus the Redis-error branches in `cache.py` that only run during an outage. The circuit breaker itself has its own unit tests.

How does the worker know when to check? Every 30 seconds it selects the active monitors whose interval has passed since their last check, probes each one, records the result, updates the status and opens or closes incidents, then invalidates the cache.

How would this scale? Add more worker containers, each covering a slice or a region; the API doesn't change. The `checks` table is the one that grows (about 1,440 rows per monitor per day), so retention and downsampling are the next step, and they're documented as known debt.

What would you build next? The open Should-haves: email alerts (US-19) and incident acknowledgement in the UI (US-18). Then Slack alerts (US-20), and a database migration tool (Alembic) before the first schema change.

What was the hardest bug? The dashboard showing "0 open incidents" while a monitor was down. The incident feed was capped at the newest 100 rows, and an old open incident had fallen off the end. The fix was to fetch open incidents separately, and regression test R1 guards it.

Part 10 — If something goes wrong

Problem	Fix
Docker Desktop crashes on start ("dockerInference … cannot be accessed")	Quit Docker. In File Explorer, rename `%LOCALAPPDATA%\Docker\run` to `run.old`, then start Docker again
`port is already allocated` (3000 or 8000)	`Get-NetTCPConnection -LocalPort 3000` shows the process. Stop it, or restart the laptop
Charts look empty, or timestamps say "84d ago"	Run `scripts\demo-reset.ps1` (fresh seed with 24 h of history)
Everything shows DOWN	No internet: the monitors probe real websites. Use a phone hotspot, or say `"this is exactly what an outage looks like"`
The broken monitor isn't DOWN yet	It needs two cycles (about 1–2 min). Move on to Part 6 and come back
The web page is slow on first load	It's the first compile in dev mode. Refresh once; `demo-reset` warms it up
A test fails live	Show the CI run on GitHub (tab 6) and the screenshots in the report, section 1
Nothing works at all	Present from the PDFs: every screen and test run is in the report

Demo login: `demo@vigil.dev` / `vigil-demo-2026` · **Repo:** github.com/vatsal-agra/vigil